Note: A summary is provided at Page 12. It is recommended to go through the summary before reading the paper

Lightning OPD: Efficient Post-Training for Large Reasoning Models with Offline On-Policy Distillation

Yecheng Wu, Song Han, Han Cai

NVIDIA

arXiv:2604.13010v1 [cs.LG] 14 Apr 2026

Abstract: On-policy distillation (OPD) has emerged as an efficient post-training paradigm for large language models, where a student is trained to match a stronger teacher's token-level distribution using dense per-token advantage signals. However, standard OPD requires a live teacher inference server throughout training, resulting in substantial infrastructure overhead. In this work, we investigate whether on-policy distillation can be performed offline. A natural approach is to precompute teacher log-probabilities once over SFT rollouts and reuse them during training. In practice, however, this offline variant fails to reliably match the performance of standard OPD. To understand this discrepancy, we identify a previously overlooked condition that is critical for any OPD pipeline, which we term **teacher consistency**. This condition requires that the same teacher model be used for both supervised fine-tuning and OPD. We show that violating teacher consistency introduces an irreducible gradient bias, causing both offline and online OPD to converge to a suboptimal fixed point regardless of training duration. Building on this insight, we propose **Lightning OPD**, an offline on-policy distillation framework that enforces teacher consistency by precomputing teacher log-probabilities over SFT rollouts. This design eliminates the need for a live teacher server entirely. We further show that, under teacher consistency, Lightning OPD shares the same optimum as standard OPD, with bounded gradient discrepancy and an implicit regularization effect that helps prevent policy drift. Extensive experiments on mathematical reasoning and code generation demonstrate that Lightning OPD achieves state-of-the-art performance with significantly improved efficiency. Starting from an SFT-initialized Qwen3-8B-Base model, Lightning OPD reaches **69.9% on AIME 2024** in just **30 GPU hours**, achieving a **4.0×** speedup over standard OPD and substantially lowering the barrier to entry for academic research on LLM post-training.

The problems with standard OPD, and the proposals by the authors

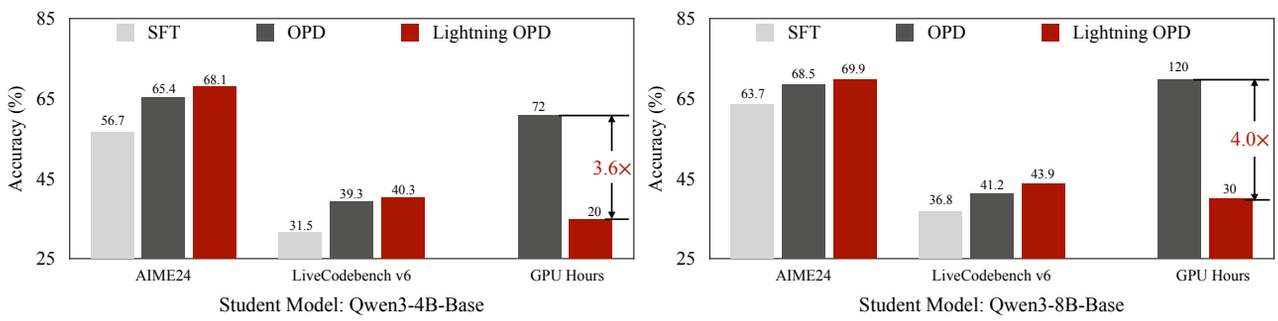


Figure 1 | Performance (Pass@1, %) and training cost of Lightning OPD compared to standard OPD and the SFT baseline on Qwen3-4B-Base and Qwen3-8B-Base models. Lightning OPD matches or outperforms standard OPD on both math and coding benchmarks across both scales, while eliminating the need for a live teacher server during training. Notably, at the 8B scale, Lightning OPD achieves a state-of-the-art **69.9%** on AIME 2024 in just **30 GPU hours**, delivering **4.0×** higher training efficiency than standard OPD. These results highlight the effectiveness of Lightning OPD as a highly efficient post-training framework for large reasoning models.

1. Introduction

Large language models (LLMs) have achieved remarkable progress across tasks such as mathematical reasoning, code generation, and multi-step agent planning [1, 2, 3, 4, 5]. This success is underpinned by carefully designed post-training pipelines [6], which typically consist of supervised fine-tuning (SFT) on high-quality data [7], followed by a reinforcement learning (RL) stage to elicit stronger reasoning capabilities. On-Policy Distillation

Corresponding author(s): Han Cai (hcai@nvidia.com).
© 2026 NVIDIA. All rights reserved.

(OPD) [8, 9, 10, 11, 12, 13, 14] has emerged as a particularly effective approach in this stage. It trains a student model to match a stronger teacher's token-level distribution using dense per-token advantage signals. Compared to Reinforcement Learning from Verifiable Rewards (RLVR) [15, 16, 1, 17], OPD provides richer supervision, offers greater training stability, and incurs significantly lower training costs, while achieving competitive or superior performance across a wide range of tasks [9, 18, 10, 14].

However, the very property that makes OPD effective—the requirement that the teacher score every student rollout during training—also introduces a persistent infrastructure bottleneck. Standard OPD relies on a dedicated multi-GPU teacher server running in parallel with the training job, leading to substantial compute overhead and making large-scale experiments costly and difficult to reproduce, particularly for academic researchers without access to extensive serving infrastructure.

A natural question is whether the benefits of on-policy supervision can be preserved while eliminating the need for a live teacher server. On-policy training is defined by the student's current rollout distribution, which evolves at every gradient step, making the teacher appear indispensable. However, recent empirical studies suggest that RL-trained models remain surprisingly close to their SFT initialization: reasoning trajectories in RL models are largely a reweighted subset of those present in the SFT model [19], and on-policy updates are inherently biased toward solutions that minimize KL divergence from the reference policy [20]. We observe a similar phenomenon in OPD training, where the student's distribution exhibits only modest drift from the SFT reference throughout the OPD stage. This observation suggests a practical alternative: precompute the teacher's log-probabilities once over SFT rollouts prior to training and reuse these values throughout the OPD process, thereby eliminating the need for a live teacher server.

This should be no surprise in 2026

In practice, however, naively applying this offline precomputation fails to reliably match the performance of standard OPD. Investigating the root cause, we find that the issue does not primarily stem from the offline approximation itself, but rather from a more fundamental condition that has been overlooked in prior OPD work, which we term ***teacher consistency***. Unlike RLVR, where model behavior is shaped solely by a reward signal, OPD involves *two* distinct teachers: one used during the SFT stage to generate training trajectories, and another used during the OPD stage to provide the reference distribution. Teacher consistency requires these two teachers to be the same model. In practice, existing pipelines often violate this condition by following conventions inherited from RLVR, where SFT datasets are curated using whichever teacher produces the highest-quality demonstrations, without regard to the teacher used during OPD. For example, Thinking Machines Lab [9] trains a Qwen3-8B-Base model on OpenThoughts-3 [7], whose trajectories are generated by QwQ-32B, while using Qwen3-32B as the OPD teacher—a mismatch that our analysis predicts to be detrimental. We show that such teacher inconsistency introduces an irreducible bias into the training objective, causing both offline and online OPD to converge to a suboptimal fixed point regardless of training duration or data scale. This issue is further exacerbated in the offline setting, where the fixed rollout distribution additionally reflects the incorrect teacher. These findings establish teacher consistency as a necessary condition for any OPD pipeline, rather than a constraint specific to the offline setting.

Why naively making the teacher offline does not work?

With teacher consistency established as the key condition, we propose **Lightning OPD** (**L**ightning **O**n-**P**olicy **D**istillation), an offline distillation framework that naturally arises from enforcing this principle. In the SFT stage, the base model is fine-tuned on trajectories generated by a chosen teacher π_T to obtain the reference policy π_{ref}. In the OPD stage, rollouts are sampled from π_{ref}, and the same teacher's log-probabilities are precomputed once over these fixed responses, eliminating the need for a live teacher server during training. We provide a rigorous theoretical analysis showing that, under teacher consistency, Lightning OPD provably shares the same optimum as standard OPD. Moreover, the gradient discrepancy between the two remains bounded throughout training, and the offline objective introduces an implicit regularization effect that naturally prevents policy drift without requiring any explicit penalty.

We evaluate Lightning OPD on mathematical reasoning and code generation tasks across diverse student–teacher model pairs, including Qwen3-4B with Qwen3-8B and Qwen3-8B with Qwen3-32B as the teacher [18]. Lightning OPD matches or outperforms standard OPD across all benchmarks while achieving approximately 4.0× speedup by removing the need for parallel teacher serving infrastructure. Concretely, starting from an SFT-initialized Qwen3-8B-Base model, Lightning OPD reaches **69.9% on AIME 2024** in just **30 GPU hours**, achieving **4.0**× higher training efficiency than standard OPD and significantly lowering the barrier for academic research on LLM post-training.

Our main contributions are summarized as follows:

- We identify *teacher consistency* as a necessary condition for effective OPD, requiring that the SFT teacher and the OPD reference teacher be the same model. We show that violating this condition introduces an irreducible gradient bias, causing both online and offline OPD to converge to a suboptimal fixed point. This issue is further exacerbated in the offline setting.
- We propose Lightning OPD, an offline distillation framework that enforces teacher consistency and eliminates the need for a live teacher server by precomputing teacher log-probabilities once over SFT rollouts. We prove that, under teacher consistency, Lightning OPD shares the same optimum as standard OPD, with bounded gradient discrepancy and an implicit regularization effect that naturally prevents policy drift.
- We validate Lightning OPD on mathematical reasoning and code generation tasks across diverse student–teacher pairs. Starting from an SFT-initialized Qwen3-8B-Base model, Lightning OPD achieves **69.9% on AIME 2024** in just **30 GPU hours**, delivering **4.0×** higher training efficiency than standard OPD and significantly lowering the barrier for academic research on LLM post-training.

2. Related Work

LLM Post-Training. Post-training has become an essential stage in developing capable LLMs, typically comprising supervised fine-tuning (SFT) on high-quality demonstrations [6, 7, 21] and a reinforcement learning (RL) stage to further improve model behavior. RL methods broadly fall into two categories, outcome-based approaches [22, 23, 24, 25, 26, 27] that optimize against sparse verifiable reward signals such as GRPO [16] and DAPO [17], and process-based approaches [28, 29, 30, 31] that provide dense per-step supervision such as PPO [15], together driving much of the recent progress in open-source reasoning models [1, 32, 33, 2, 3, 4, 5, 34, 18, 35]. A growing line of work recognizes that the SFT and RL stages are not independent and explores their co-design [36, 37, 38, 39, 40, 41]. Our work proposes a new post-training framework based on on-policy distillation and contributes a co-design perspective from the OPD angle, showing that the SFT and OPD stages must share the same teacher to guarantee effective training and establishing teacher consistency as a principled constraint on how the two stages should be jointly designed.

On-Policy Distillation. Knowledge distillation [42] transfers capabilities from a large teacher to a smaller student by minimizing their output divergence. Standard offline KD trains on fixed teacher-generated data [43, 44], while on-policy distillation (OPD) samples trajectories from the student and aligns it with the teacher's token-level distribution on these student-generated rollouts [8, 9], achieving faster and more effective distillation than offline counterparts [18, 10]. Recent OPD work has explored reward extrapolation to push the student beyond the teacher's own capability [10], black-box variants that do not require access to teacher logits [45], self-distillation paradigms that leverage the model's own in-context capabilities [11, 13, 12], controllable multi-budget reasoning via on-policy exploration [46], and privileged information distillation that transfers training-time knowledge to inference-time policies [47]. Concurrent work further validates OPD across diverse post-training tasks [14]. Building on this line of work, our Lightning OPD advances OPD toward greater efficiency by eliminating the live teacher server requirement without sacrificing the dense per-token supervision that makes OPD effective. A key enabler of this design is what we identify as *teacher consistency*, the requirement that the SFT-stage and OPD-stage teachers be the same model. Existing OPD pipelines routinely ignore this condition. For example, Thinking Machines Lab [9] performs SFT on OpenThoughts-3 [7] trajectories generated by QwQ-32B while using Qwen3-32B as the OPD teacher. We show that this mismatch introduces an irreducible gradient bias that degrades both standard OPD and Lightning OPD, establishing teacher consistency as a principled design guideline for the broader OPD practice.

Off-Policy Reinforcement Learning. Off-policy RL decouples data collection from policy optimization, enabling reuse of past experience with advantages in sample efficiency and training stability [48, 49]. These benefits have motivated a rich literature on offline RL methods [50, 51, 52] and their application to LLM alignment [53, 54, 55], as well as more recent work on off-policy reasoning that handles distributional mismatch through importance sampling corrections [15, 16], designs algorithms tolerant of substantial policy lag [56], or trains on fixed datasets for superior stability [57]. Lightning OPD shares the structural form of optimizing over a fixed dataset, but is not a straightforward application of offline RL to OPD. The core challenges of offline RL, including OOD action overestimation and sparse reward variance, do not apply here because the

teacher provides dense token-level supervision for all sequences. We discuss the precise relationship between Lightning OPD and offline RL in Section 3.4.

3. Methodology

3.1. Preliminaries

Let π_T denote a fixed teacher model and π_θ a trainable student. Given a prompt q, a response $x = (a_1, \ldots, a_T)$ is generated autoregressively as

$$\pi_\theta(x \mid q) = \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t), \tag{1}$$

where $s_t = (q, a_1, \ldots, a_{t-1})$. Let π_{ref} denote the SFT-initialized student. The per-token OPD advantage is

$$\boxed{A_t(\theta) = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t),} \tag{2}$$

which is positive where the teacher is more confident than the student and negative otherwise. Standard OPD [8, 9, 18] optimizes

$$J_{\text{on}}(\theta) = \mathbb{E}_{q\sim p,\ x\sim\pi_\theta}\left[\sum_{t=1}^{T} A_t(\theta)\right], \tag{3}$$

while Lightning OPD fixes the rollout distribution to π_{ref} and optimizes

$$J_{\text{off}}(\theta) = \mathbb{E}_{q\sim p,\ x\sim\pi_{\text{ref}}}\left[\sum_{t=1}^{T} A_t(\theta)\right]. \tag{4}$$

Notice the difference: Lightning opd samples from the reference policy, not the current policy

Both objectives share the same advantage function and differ only in the response distribution. By an IS decomposition, $\nabla J_{\text{on}}(\theta) = \mathbb{E}_{x\sim\pi_{\text{ref}}}[w(x;\theta)\cdot\nabla J_{\text{off}}(\theta)]$ where $w(x;\theta) = \pi_\theta(x)/\pi_{\text{ref}}(x)$, so the offline gradient is the special case $w \equiv 1$.

3.2. Lightning On-Policy Distillation

Following the common practice of LLM post-training [6, 1], Lightning OPD consists of two stages. We describe each stage in detail below and highlight where Lightning OPD departs from standard OPD.

Stage 1: Supervised Fine-Tuning. Given a base model π_{base}, a teacher π_T, and a prompt dataset $\mathcal{Q}_{\text{SFT}}$, we collect a supervised dataset from teacher-generated trajectories:

$$\mathcal{D}_{\text{SFT}} = \left\{ (q^i,\ x^i) \ \middle|\ x^i \sim \pi_T(\cdot \mid q^i),\ q^i \in \mathcal{Q}_{\text{SFT}} \right\}. \tag{5}$$

Responses Sampled from teacher

The base model is then fine-tuned on $\mathcal{D}_{\text{SFT}}$ via maximum likelihood estimation to obtain the reference policy π_{ref}:

$$\pi_{\text{ref}} = \arg\max_{\theta}\ \mathbb{E}_{(q,x)\sim\mathcal{D}_{\text{SFT}}}\left[\sum_{t=1}^{T} \log \pi_\theta(a_t \mid s_t)\right]. \tag{6}$$

SFT base student model on the above dataset where responses are generated by teacher

Main difference between standard OPD and current method

While standard OPD may use any high-quality SFT dataset regardless of its source, Lightning OPD requires that $\mathcal{D}_{\text{SFT}}$ be generated by the same teacher π_T used in the OPD stage. This *teacher consistency* condition is a prerequisite for the offline approximation to be sound, as we formally establish in Section 3.

Stage 2: Offline On-Policy Distillation. Starting from a task-specific prompt dataset $\mathcal{Q}_{\text{OPD}}$, the OPD stage proceeds in two phases. In the **preprocessing phase**, rollouts are sampled from π_{ref} and the teacher is queried *once* to precompute and store per-token log-probabilities, forming the offline dataset:

$$\mathcal{D}_{\text{OPD}} = \left\{ (q^j,\ x^j,\ \{\log \pi_T(a_t^j \mid s_t^j)\}_{t=1}^{T_j}) \ \middle|\ x^j \sim \pi_{\text{ref}}(\cdot \mid q^j),\ q^j \in \mathcal{Q}_{\text{OPD}} \right\}. \tag{7}$$

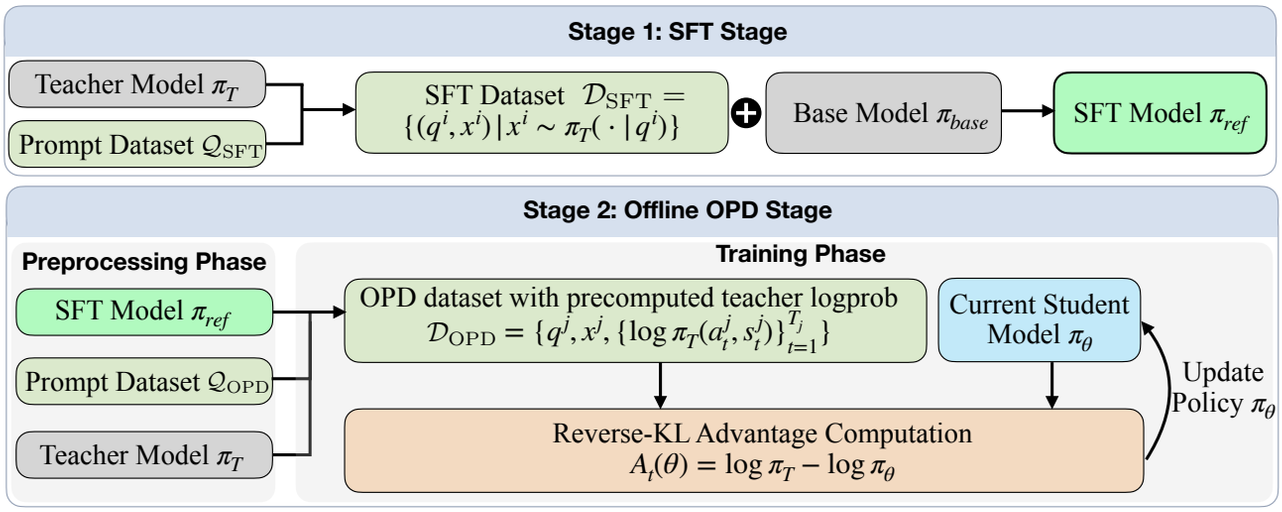


Figure 2 | Overview of Lightning OPD. In the SFT stage, the teacher π_T generates trajectories on $\mathcal{Q}_{\text{SFT}}$ then the base model π_{base} is fine-tuned on these trajectories to obtain the SFT model π_{ref}. OPD stage proceeds in two phases. In the preprocessing phase, rollouts are sampled from π_{ref} on $\mathcal{Q}_{\text{OPD}}$ and the teacher is queried once to compute and store per-token log-probabilities $\log \pi_T(a_t \mid s_t)$, forming the offline dataset $\mathcal{D}_{\text{OPD}}$. In the training phase, the student π_θ is initialized from π_{ref} and trained on $\mathcal{D}_{\text{OPD}}$. At each step, the per-token advantage $A_t = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t)$ is computed by reading $\log \pi_T$ from $\mathcal{D}_{\text{OPD}}$ while computing $\log \pi_\theta$ online. During this process, we do not need a live teacher model. In contrast, standard OPD requires a live teacher server throughout training to compute advantages at every gradient step. By eliminating this requirement, Lightning OPD reduces total training cost of OPD by **4.0**× at the 8B scale (from 120 to 30 GPU hours).

In contrast, standard OPD requires a live teacher server throughout training, querying it at every step for log-probabilities on freshly sampled rollouts from the current policy π_θ. In the **training phase**, the student is initialized from π_{ref} and trained on $\mathcal{D}_{\text{OPD}}$ with no teacher server required. At each step, the advantage is computed as $A_t(\theta) = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t)$, where the teacher term is read directly from $\mathcal{D}_{\text{OPD}}$ and the student term is computed online. This eliminates the primary infrastructure bottleneck of standard OPD and makes high-quality on-policy distillation accessible with minimal compute overhead. The full procedure is given in Algorithm 1.

3.3. Theoretical Analysis

We characterize when and why Lightning OPD matches standard OPD and identify the precise conditions under which this equivalence holds. All proofs are deferred to Appendix A. The analysis rests on three standard assumptions, with a fourth introduced for the teacher-mismatch analysis.

Assumption 3.1 (Bounded Second Moment of Advantage)**.** There exists $\sigma_A < \infty$ such that for all $\theta \in \Theta$: $\mathbb{E}_{x \sim \pi_{\text{ref}}}\left[\left(\sum_{t=1}^{T} A_t(\theta)\right)^2\right] \leq \sigma_A^2$, where $A_t(\theta) = \log \pi_T(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t)$.

Assumption 3.2 (Support Coverage)**.** For all θ encountered during optimization and all prompts q: $\text{supp}(\pi_\theta(\cdot \mid q)) \subseteq \text{supp}(\pi_{\text{ref}}(\cdot \mid q))$.

Assumption 3.3 (Bounded Score Function)**.** There exists $G < \infty$ such that $\|\nabla \log \pi_\theta(a_t \mid s_t)\|_2 \leq G$ for all $\theta \in \Theta$ and all t.

Assumption 3.4 (Bounded Teacher Mismatch)**.** Let $\Delta_t = \log \pi_T^{\text{SFT}}(a_t \mid s_t) - \log \pi_T^{\text{OPD}}(a_t \mid s_t)$. There exists $\sigma_\Delta < \infty$ such that $\mathbb{E}_{x \sim \pi_{\text{ref}}}\left[\left(\sum_{t=1}^{T} \Delta_t\right)^2\right] \leq \sigma_\Delta^2$. When teacher consistency holds ($\pi_T^{\text{SFT}} = \pi_T^{\text{OPD}}$), $\Delta_t = 0$ everywhere and $\sigma_\Delta = 0$.

Assumptions 3.1–3.3 are standard: 3.1 is an L^2 condition that avoids vacuous sup-norm bounds, 3.2 holds naturally when π_θ is initialized from π_{ref}, and 3.3 is standard in policy gradient analyses. Assumption 3.4 quantifies the teacher mismatch and is only needed for the teacher consistency results (Theorems 3.11–3.13).

Algorithm 1 Lightning On-Policy Distillation (Lightning OPD)

Require: Base model π_{base}, teacher π_T, prompt datasets $\mathcal{Q}_{\text{SFT}}$, $\mathcal{Q}_{\text{OPD}}$, learning rate η

```
// Stage 1: SFT
Collect D_SFT = {(q, x) | q ∈ Q_SFT, x ~ π_T(· | q)}
Fine-tune π_base on D_SFT to obtain π_ref
// Stage 2, Phase 1: Preprocessing
for each prompt q^j ∈ Q_OPD do
    Sample x^j ~ π_ref(· | q^j); store log π_T(a_t^j | s_t^j) for all t
end for
Form D_OPD = {(q^j, x^j, {log π_T(a_t^j | s_t^j)}_t)}
// Stage 2, Phase 2: Training
Initialize π_θ ← π_ref
for each training step do
    Sample mini-batch from D
    Compute A_t(θ) = log π_T(a_t | s_t) − log π_θ(a_t | s_t), clip to [−τ, τ]
    Update θ ← θ + η · ∇J_off(θ)
end for
```

For the first time, i think you do not need any math to justify this algorithm. This is more like another experiment if you have time!

Theorem 3.5 (Gradient Discrepancy Bound)**.** Under Assumptions 3.1–3.3,

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 \le G \cdot \sigma_A \cdot \sqrt{\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}})}, \tag{8}$$

where G bounds the score function norm, σ_A is the L^2 norm of the cumulative advantage under π_{ref}, and $\chi^2(\pi_\theta \| \pi_{\text{ref}}) = \mathbb{E}_{x\sim\pi_{\text{ref}}}[w(x;\theta)^2] - 1$.

Remark 3.6. At initialization $\pi_\theta = \pi_{\text{ref}}$, we have $\chi^2 = 0$ and the two gradients coincide exactly. The bound grows as π_θ drifts from π_{ref}, but remains small under standard KL regularization [15, 17], which keeps the student near its initialization throughout training.

Despite this gradient gap, both objectives share the same performance optimum.

Theorem 3.7 (Shared Fixed Point)**.** The stationary conditions of J_{on} and J_{off} over Θ coincide. Every stationary point θ^* minimizes $\text{KL}(\pi_\theta \| \pi_T)$ over Π_Θ and is shared by both standard OPD and Lightning OPD.

Remark 3.8. The performance ceiling of Lightning OPD is determined solely by model capacity Π_Θ relative to the teacher, not by the choice of rollout distribution. Switching to online rollouts can only reduce the distribution-shift error bounded in Theorem 3.5, but cannot lower the irreducible approximation error $\varepsilon_{\text{approx}} = \min_\theta \text{KL}(\pi_\theta \| \pi_T)$.

Moreover, the offline objective itself resists the drift that causes the bound in Theorem 3.5 to grow.

Theorem 3.9 (Implicit Regularization)**.** The offline gradient decomposes as

$$\nabla J_{\text{off}}(\theta) = \nabla J_{\text{on}}(\theta) - \text{Cov}_{\pi_{\text{ref}}}[w(x;\theta),\ f(x;\theta)]\,, \tag{9}$$

where $f(x;\theta) = \sum_t A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)$.

Remark 3.10. The covariance term vanishes when $\pi_\theta = \pi_{\text{ref}}$ and grows as the student diverges, acting as an implicit trust-region penalty that opposes drift from π_{ref}. Unlike an explicit KL penalty, this regularization requires no hyperparameter and arises automatically from fixing the rollout distribution.

The three results above assume the SFT and OPD stages use the same teacher. We formalize this as *teacher consistency* and show that violating it introduces an irreducible bias in both paradigms.

Theorem 3.11 (Teacher Consistency and the Offline-Online Gap)**.** Let π_T^{SFT} and π_T^{OPD} denote the teachers used in the SFT and OPD stages. Under Assumptions 3.1–3.4,

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 \le G \cdot \left(\sigma_A \sqrt{\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}})} + \sigma_\Delta\right). \tag{10}$$

When $\sigma_\Delta > 0$, the additive term $G\sigma_\Delta$ is irreducible and persists even at initialization.

Remark 3.12. The bias σ_Δ arises because π_{ref} concentrates mass on trajectories preferred by π_T^{SFT}, while the advantage reflects π_T^{OPD}. This mismatch injects a persistent gradient error into Lightning OPD that no amount of additional training can remove. The mismatch constant σ_Δ is directly controlled by the divergence between the two teachers and vanishes exactly when they coincide.

Theorem 3.13 (Teacher Consistency and Standard OPD)**.** Let $\nabla J_{\text{on}}^{\delta=0}(\theta)$ denote the standard OPD gradient under a consistent teacher ($\pi_T^{\text{SFT}} = \pi_T^{\text{OPD}}$). Then at any θ reachable from π_{ref},

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{on}}^{\delta=0}(\theta)\|_2 \leq G \cdot \sigma_\Delta. \tag{11}$$

Standard OPD therefore also converges to a suboptimal fixed point when $\sigma_\Delta > 0$.

Remark 3.14. Theorems 3.11–3.13 together show that $G\sigma_\Delta$ corrupts the gradient of *both* paradigms equally, independent of whether rollouts are online or offline. Teacher consistency is therefore a fundamental requirement shared by Lightning OPD and standard OPD. When $\sigma_\Delta = 0$ (i.e., the two teachers are identical), Theorems 3.5–3.7 apply in full and Lightning OPD provably matches standard OPD.

3.4. Discussion

We discuss how Lightning OPD relates to two superficially similar paradigms, offline RL and offline knowledge distillation, and clarify why neither subsumes our approach.

Relation to Offline RL. Lightning OPD resembles offline RL in that both optimize over a fixed dataset, but the resemblance is superficial and a direct application of offline RL techniques to OPD would fail for reasons that offline RL methods are not designed to address. The central challenge of offline RL is OOD action overestimation arising from sparse reward signals, which offline RL methods address through conservatism mechanisms such as value pessimism or policy constraint. Neither problem exists in Lightning OPD, where the teacher supplies dense per-token log-probability supervision for all token sequences, leaving no OOD region and no sparse reward to cause high-variance estimation. Conservatism is therefore neither necessary nor applicable. The real obstacle to offline OPD is teacher inconsistency. When $\sigma_\Delta > 0$, the irreducible bias $G\sigma_\Delta$ is a structural property of the gradient field itself, not an estimation artifact from limited data coverage, and no importance sampling correction or conservatism mechanism can remove it. Beyond the challenge, the two paradigms also differ in the nature of their solutions. Offline RL fixed points are data-limited, shaped by the coverage of the behavior policy, so better data coverage leads to better policies. Theorem 3.7 shows that Lightning OPD's fixed point is instead capacity-limited, determined solely by model capacity Π_Θ relative to the teacher and independent of the rollout distribution. Improving the rollout distribution cannot push past the teacher's own capability, and the right lever is model capacity, not data coverage. In summary, Lightning OPD is not a variant of offline RL applied to distillation. It is a principled offline approximation to an on-policy distillation objective, one whose unique challenge is teacher inconsistency rather than distributional coverage, and one that provably recovers standard OPD exactly when teacher consistency is enforced.

Relation to Offline Knowledge Distillation. Lightning OPD also differs fundamentally from offline (off-policy) knowledge distillation [43, 44], despite the shared use of precomputed teacher signals. Offline KD trains the student on teacher-generated sequences, so the student only receives supervision on trajectories the teacher would produce, never on its own mistakes. Lightning OPD instead collects rollouts from the student's own policy π_{ref} and evaluates the teacher's per-token log-probabilities on these student-generated sequences. This means the teacher provides corrective signals on exactly the distribution the student will encounter during inference, which is the core advantage of on-policy methods over off-policy ones [8]. The distinction is preserved in Lightning OPD even though teacher log-probabilities are precomputed. Empirically, the OPD stage provides substantial gains over the SFT baseline across all benchmarks, confirming that on-policy supervision on student rollouts extracts significantly more from the teacher than offline KD on teacher-generated data alone. Theorem 3.7 explains this gap by showing that the fixed point of Lightning OPD is capacity-limited, depending on how well π_θ can approximate π_T across the full sequence space, rather than being restricted to the support of teacher-generated data.

4. Experiments

4.1. Experimental Setup

Models. We train two student models under the Lightning OPD pipeline, covering different model scales from the Qwen3 model family [18]. The first uses Qwen3-4B-Base as the student with Qwen3-8B as the teacher. The second uses Qwen3-8B-Base as the student with Qwen3-32B as the teacher. Both pipelines follow the two-stage procedure described in Section 3.2. The base model is first fine-tuned on teacher-generated trajectories to obtain π_{ref}, which is then used to sample rollouts and precompute teacher log-probabilities for the OPD stage.

Training Data. The SFT stage uses prompts from OpenThoughts-3 [7], with responses generated by the respective teacher model. For the OPD stage, we train on two domains. Mathematical reasoning uses DAPO-Math-17k [17], which provides 17K competition-level math problems spanning a wide range of difficulty. Code generation uses a sampled 30K subset of EpiCoder-func-380k [58], which provides diverse function-level code synthesis problems. For each prompt, we sample a single response from π_{ref} and precompute the corresponding teacher log-probabilities once prior to training, with no teacher server required during the OPD stage.

Benchmarks. For mathematical reasoning, we evaluate on AIME 2024 [59], AIME 2025 [60], and HMMT 2025 [61]. For code generation, we evaluate on LiveCodeBench v5 and v6 [62]. In all evaluations, we set the temperature to 0.6, top-p to 0.95, and the maximum generation length to 32,768 for math benchmarks and 40,960 for coding benchmarks. We sample 32 solutions per problem for math benchmarks and 4 solutions per problem for code benchmarks, reporting the average pass@1.

Training Settings. The SFT stage is implemented using LlamaFactory [63] and the OPD stage is implemented using slime [64]. The OPD stage is trained for 150 steps, which we find sufficient for convergence as shown in Figure 3b. Standard OPD and Lightning OPD share identical training settings in the OPD stage, differing only in the source of rollouts that standard OPD samples rollouts online from the current student while Lightning OPD reuses rollouts precomputed from π_{ref} before training begins. Full hyperparameter details are provided in Appendix B.

4.2. Main Results

Table 1 presents evaluation results across five benchmarks for both 4B and 8B model scales. The central finding is that Lightning OPD, despite eliminating the live teacher server entirely during training, achieves performance on par with standard OPD across all settings, and in several cases marginally exceeds it. This validates our theoretical analysis that under teacher consistency, the offline approximation preserves the same performance optimum as standard OPD. The gains from the OPD stage over the SFT baseline are substantial and consistent across both math and code benchmarks, confirming that on-policy distillation provides strong and transferable post-training improvements. At the 4B scale, compared to ExOPD [10], a recent OPD baseline, Lightning OPD achieves substantially better results, reaching 68.1% compared to 61.0% on AIME 2024, and the gap widens on code generation where Lightning OPD reaches 40.3% on LCB v6 against ExOPD's 29.0%. At the 8B scale, Lightning OPD achieves 69.9% on AIME 2024 and 49.5% on LiveCodeBench v5. Together, these results demonstrate the effectiveness of Lightning OPD as a general and efficient post-training framework across model scales and task domains.

4.3. Training Cost

Table 2 compares the training cost of standard OPD and Lightning OPD. Lightning OPD achieves a 3.6× speedup at the 4B scale, reducing total GPU hours from 72 to just 20, and a 4.0× speedup at the 8B scale, bringing the full pipeline down from 120 to just 30 GPU hours. We also provide a per-phase breakdown of the Lightning OPD pipeline. The actual OPD training stage consumes only a small fraction of this budget, with the remaining cost split between rollout collection and teacher logprob precomputation, both of which are

Table 1 | Pass@1 on math reasoning and code generation benchmarks. Lightning OPD matches or surpasses standard OPD across all benchmarks at both model scales, while requiring no live teacher server during training. At the 4B scale, Lightning OPD substantially outperforms ExOPD [10], achieving 68.1% compared to 61.0% on AIME 2024 and 40.3% compared to 29.0% on LCB v6. At the 8B scale, Lightning OPD reaches 69.9% on AIME 2024 and 49.5% on LCB v5. **Bold** indicates the best result within each model scale.

Method	**Math Reasoning**				**Code Generation**		
	AIME 2024	**AIME 2025**	**HMMT 2025**	**Avg.**	**LCB v5**	**LCB v6**	**Avg.**
Student: Qwen3-4B-Base Teacher: Qwen3-8B							
+ SFT	56.7	52.1	34.0	47.6	33.8	31.5	32.6
+ OPD	65.4	57.9	**39.9**	54.4	**44.2**	39.3	**41.8**
+ Lightning OPD	**68.1**	**58.4**	39.8	**55.4**	42.8	**40.3**	41.5
+ ExOPD [10]	61.0	56.0	34.4	50.5	–	29.0	–
Student: Qwen3-8B-Base Teacher: Qwen3-32B							
+ SFT	63.7	51.7	36.9	50.8	44.7	36.8	40.8
+ OPD	68.5	59.0	39.4	55.6	47.3	41.2	44.2
+ Lightning OPD	**69.9**	**59.2**	**41.9**	**57.0**	**49.5**	**43.9**	**46.7**

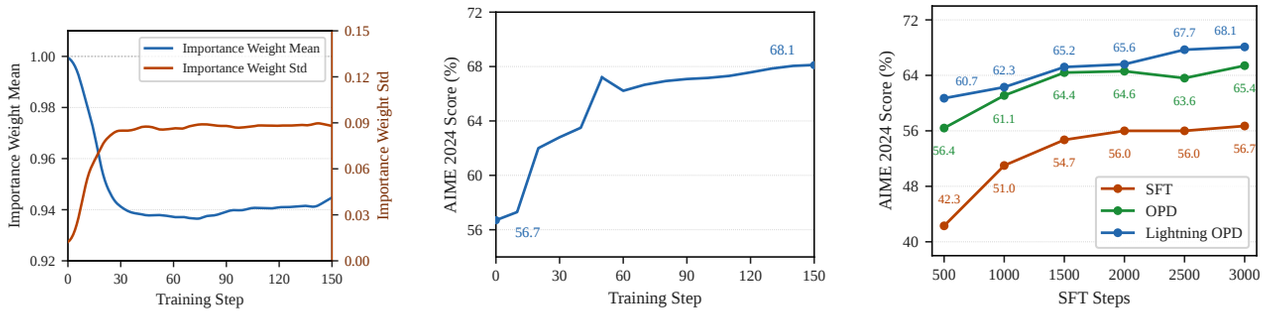
Table 2 | Training cost (GPU hours) of standard OPD vs. Lightning OPD at the 4B and 8B scales. Lightning OPD achieves a 3.6× speedup at 4B and a 4.0× speedup at 8B, requiring only 20 and 30 GPU hours to train a reasoning model respectively. The lower panel breaks down Lightning OPD costs by stage, showing that the actual OPD training consumes only a moderate fraction of the total budget, highlighting how the minimal infrastructure requirement of Lightning OPD makes the overall post-training pipeline highly efficient.

Method	*Qwen3-4B-Base*	*Qwen3-8B-Base*
OPD	72	120
Lightning OPD	20	30
Speedup	**3.6**×	**4.0**×
Lightning OPD breakdown		
∟ Rollout collection	10	10
∟ Teacher logprob precompute	2	4
∟ OPD training	8	16

one-time offline operations that require no specialized infrastructure. This stands in sharp contrast to standard OPD, which demands a dedicated multi-GPU teacher server running continuously throughout training. The minimal infrastructure requirement of Lightning OPD makes high-quality on-policy distillation accessible to practitioners without large-scale training and serving systems.

4.4. Training Dynamics

Figure 3 examines the internal dynamics of Lightning OPD across both stages of training. Figure 3a tracks the per-token importance weight $w_t = \pi_\theta/\pi_{\text{ref}}$ throughout OPD training. The mean drops from 1 to 0.94 within the first 20 steps and then plateaus, while the standard deviation rises sharply in the same early phase before stabilizing at a moderate level. The mean remaining close to 0.94 indicates that the student policy stays near the reference distribution throughout training, and the standard deviation stabilizing below 0.1 indicates that per-token distributional shift is consistently small. Both quantities remaining bounded confirms the implicit regularization of Theorem 3.9, which shows that fixing rollouts to π_{ref} automatically constrains both the magnitude and spread of policy drift without any explicit KL penalty. Figure 3b shows the AIME 2024 score throughout OPD training. The score converges remarkably fast, with the student capturing nearly all of its performance gain within the first 50 steps and remaining stable thereafter, justifying our choice of 150 steps as a sufficient training budget. Figure 3c shows the effect of SFT checkpoint quality on the final model.



(a) Importance weight dynamics. (b) AIME 2024 score vs. training step. (c) AIME 2024 score vs. SFT steps.

Figure 3 | Training dynamics of Lightning OPD (Qwen3-4B-Base student). **(a)** The mean importance weight $w_t = \pi_\theta/\pi_{\text{ref}}$ drops rapidly to ≈ 0.94 within the first 20 steps and plateaus, with its standard deviation stabilizing concurrently, validating the implicit regularization in Theorem 3.9. **(b)** AIME 2024 pass@1 rises steeply in the first 50 OPD steps and saturates thereafter, confirming that 150 steps is a sufficient training budget. **(c)** AIME 2024 pass@1 of the SFT model, standard OPD, and Lightning OPD as a function of SFT checkpoint quality. All three improve consistently with more SFT steps, and both OPD variants provide a large, stable gain on top of the SFT baseline at every checkpoint.

Table 3 | Teacher consistency ablation for standard OPD (AIME 2024 pass@1, %). Rows indicate the SFT-stage teacher and columns indicate the OPD-stage teacher. Using the same teacher in both stages (**diagonal, bold**) always achieves the best performance. Inconsistent teacher pairs degrade accuracy by 2–4 points, confirming Theorems 3.11–3.13.

	OPD Teacher			OPD Teacher	
SFT Teacher	Qwen3-8B	QwQ-32B	SFT Teacher	Qwen3-32B	QwQ-32B
Student: Qwen3-4B-Base			*Student: Qwen3-8B-Base*		
Qwen3-8B	**65.4**	62.4	Qwen3-32B	**68.5**	64.8
QwQ-32B	61.2	**62.8**	QwQ-32B	65.0	**66.5**

All three curves improve consistently with more SFT steps, and both OPD variants provide a large, stable gain on top of the SFT baseline at every checkpoint. The relative ordering of the three methods remains consistent across all SFT budgets, indicating that Lightning OPD is robust to the choice of SFT training length.

4.5. Ablation Study

Teacher consistency. To empirically verify the teacher consistency principle, we design an ablation that crosses SFT-stage and OPD-stage teacher choices. We introduce QwQ-32B [65] as an additional teacher alongside Qwen3-32B at the 8B scale, and QwQ-32B alongside Qwen3-8B at the 4B scale, yielding a full grid of SFT and OPD teacher combinations. All other settings follow the main experiments. We conduct this ablation for both standard OPD and Lightning OPD, reported in Tables 3 and 4 respectively. The results uniformly confirm our theoretical prediction that the teacher-consistent setting, on the diagonal, always achieves the best performance, while mismatched teachers consistently degrade accuracy across all model scales and both paradigms. Notably, teacher inconsistency imposes a larger penalty on Lightning OPD than on standard OPD. At the 8B scale, mismatching from Qwen3-32B SFT to QwQ-32B OPD drops Lightning OPD by 6.8 points, from 69.9% to 63.1%, but drops standard OPD by only 3.7 points, from 68.5% to 64.8%. A similar asymmetry appears in the reverse direction. Averaged across all inconsistent teacher combinations and both model scales, Lightning OPD falls 1 to 2 points below standard OPD. This is precisely the failure mode we identified, namely that naively applying offline precomputation without enforcing teacher consistency does not reliably match standard OPD, and the gap is not random noise but a systematic consequence of the teacher mismatch. These findings are consistent with Theorems 3.11 and 3.13, which predict that mismatched teachers introduce an irreducible gradient bias of magnitude $G\sigma_\Delta$ into both paradigms, where σ_Δ measures the teacher mismatch under π_{ref} (Assumption 3.4).

Table 4 | Teacher consistency ablation for Lightning OPD (AIME 2024 pass@1, %). Rows indicate the SFT-stage teacher and columns indicate the OPD-stage teacher. The consistent setting (**diagonal, bold**) is optimal in all cases. Teacher inconsistency degrades Lightning OPD more severely than standard OPD (up to 7 points at 8B), because the fixed rollout distribution compounds the gradient bias when the SFT and OPD teachers differ (Theorem 3.11).

Student: Qwen3-4B-Base			*Student: Qwen3-8B-Base*		
	OPD Teacher			OPD Teacher	
SFT Teacher	Qwen3-8B	QwQ-32B	SFT Teacher	Qwen3-32B	QwQ-32B
Qwen3-8B	**68.1**	62.5	Qwen3-32B	**69.9**	63.1
QwQ-32B	59.3	**63.1**	QwQ-32B	62.1	**68.7**

The amplified penalty for Lightning OPD arises from a compounding effect not fully captured by the per-step bound: a mismatched SFT teacher corrupts π_{ref} in two roles simultaneously, both as the reference distribution and as the source of fixed rollouts, whereas standard OPD refreshes its rollout distribution from the current student at every step and can therefore partially recover from the reference bias over the course of training. Teacher consistency is consequently a more critical design requirement for Lightning OPD than for standard OPD.

5. Conclusion

We presented Lightning OPD, an offline on-policy distillation framework for efficient LLM post-training. By precomputing teacher log-probabilities once over SFT rollouts, Lightning OPD reduces the infrastructure of OPD to a single standard training job while preserving the dense per-token supervision that makes OPD effective. We further identified teacher consistency as a necessary condition for OPD to work well in general, proving that mismatched SFT and OPD teachers introduce an irreducible gradient bias that causes both paradigms to converge to a suboptimal fixed point, and that under consistency Lightning OPD provably shares the same optimum as standard OPD. Empirically, Lightning OPD trained an 8B reasoning model to 69.9% on AIME 2024 in just 30 GPU hours, achieving 4.0× speedup over standard OPD while matching its performance across all benchmarks. We hope these results encourage broader adoption of on-policy distillation in settings where a persistent teacher server is impractical, and that the teacher consistency principle provides a useful design guideline for future OPD research.

References

[1] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[2] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

[3] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, SH Cai, Yuan Cao, Y Charles, HS Che, Cheng Chen, Guanduo Chen, et al. Kimi k2. 5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.

[4] NVIDIA. Nvidia nemotron 3: Efficient and open intelligence. *arXiv preprint arXiv:2512.20856*, 2025.

[5] Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.

[6] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[7] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.

Summary

Introduction

Most of the magic that you experience in LLMs today is because of good mid-training and post-training strategies. One such strategy that is now commonly used in post-training is on-policy distillation. It trains a student model to match a stronger teacher's token-level distribution using dense per-token advantage signals. Though OPD is very effective, the requirement that the teacher score every student rollout during training introduces a persistent infrastructure bottleneck. If you are already compute-constrained, then doing large-scale experiments with OPD becomes extremely costly. The authors tend to solve this exact problem where they want to have all the benefits of OPD, but without a live teacher to ease the compute demand.

On-Policy Distillation

Standard offline Knowledge Distillation (KD) trains on fixed teacher-generated data. On-policy distillation (OPD), on the other hand, samples trajectories from the student and aligns the student with the teacher's token-level distribution on these student-generated rollouts. Lightning OPD tries to advance the existing OPD pipeline by eliminating the need for a live teacher without sacrificing the dense per-token supervision that makes OPD effective. One of the key design aspects of Lightning OPD is the teacher-consistency (more on this later).

Off-Policy Reinforcement Learning

Off-policy RL decouples data collection from policy optimization, which is better for training stability and somewhat efficiency. Lightning OPD resembles offline RL in that both optimize over a fixed dataset, but that is the only similarity. Unlike offline RL, conservatism is therefore neither necessary nor applicable. The real obstacle to offline OPD is teacher inconsistency (explained later). Offline RL is often data-limited, and better dataset coverage generally gives better policies. Lightning OPD is capacity-limited, determined solely by model capacity relative to the teacher and independent of the rollout distribution. Improving the rollout distribution cannot push it past the teacher's own capability. The right lever here is model capacity, not data coverage.

Methodology

- Preliminaries:
  - $\pi(T)$ - A fixed teacher model
  - π(ref) - A student model SFTed on teacher-generated trajectories on some dataset
  - $\pi(\theta)$ - Trainable student model initialized from π(ref)
  - Per-token OPD advantage is given by: $At(\theta) = \log \pi T(at \mid st) - \log \pi\theta(at \mid st)$
  - It is positive when the teacher is more confident than the student, and negative when vice versa.
  - Standard OPD objective functions are given as:

$$J_{\text{on}}(\theta) = \mathbb{E}_{q\sim p,\ x\sim\pi_\theta}\left[\sum_{t=1}^{T} A_t(\theta)\right],$$

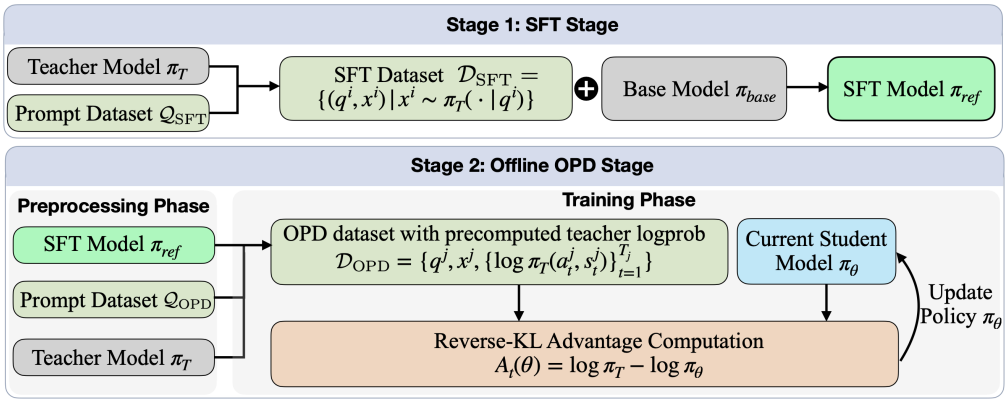
  - Lightning OPD objective function is given as:

$$J_{\text{off}}(\theta) = \mathbb{E}_{q\sim p,\ x\sim\pi_{\text{ref}}}\left[\sum_{t=1}^{T} A_t(\theta)\right].$$

  - Both objectives share the same advantage function and differ only in the response distribution

- Lightning On-Policy Distillation: Divided into two stages: SFT and offline OPD.
  - Stage 1 - SFT: Given a base model, a teacher model, and a prompt dataset, the authors first collect a supervised dataset from teacher-generated trajectories. The base model is then fine-tuned on this SFT dataset via maximum likelihood estimation to obtain the reference policy as shown below:

- Stage 2 - Offline On-Policy Distillation: This stage is also divided into two phases. The first phase is the preprocessing phase, where rollouts are sampled from the reference, and the teacher is queried once to precompute and store per-token log-probabilities, forming the offline dataset. The second phase is the training phase, where the student is initialized from the SFTed model π(ref) and trained on the OPD dataset (generated above) with no teacher server required. At each step, the advantage is computed as $At(\theta)$ = log $\pi T(at|st)$−log $\pi\theta(at|st)$, where the teacher term is read directly from the OPD dataset, and the student term is computed online.



- A major difference between other OPD strategies and Lightning-OPD is the teacher consistency. Lightning OPD ensures that we use the same teacher during the training phase, which we used during the preprocessing phase. People overlook this teacher model inconsistency, and use one teacher model to generate the SFT dataset and another (depending on the available compute) for OPD. The real obstacle to offline OPD is this inconsistency in the teacher model.

- They proved that mismatched teachers introduce an irreducible gradient bias of magnitude $G(\sigma\Delta)$, in both the standard OPD and the lightning OPD, where $\sigma\Delta$ measures the teacher mismatch under πref. The experimental results uniformly confirm their theoretical prediction that the teacher-consistent setting always achieves the best performance, while mismatched teachers consistently degrade accuracy across all model scales and both paradigms.

Experimental Setup

- Models
  - Two student models. The first uses Qwen3-4B-Base as the student with Qwen3-8B as the teacher. The second uses Qwen3-8B-Base as the student with Qwen3-32B as the teacher.
  - The base model is first fine-tuned on teacher-generated trajectories to obtain π(ref), which is then used to sample rollouts and precompute teacher log-probabilities for the OPD stage.

- Training Data
  - The SFT stage uses prompts from OpenThoughts-3 with responses generated by the respective teacher model.
  - DAPO-Math-17k and EpiCoder-func-380k were used for the OPD stage. For each prompt, the authors sample a single response from π(ref) and precompute the corresponding teacher log-probabilities once before training, eliminating the need for a live teacher server for the OPD stage.

- Benchmarks
  - AIME 2024, AIME 2025, and HMMT 2025 for mathematical reasoning.
  - LiveCodeBench v5 and v6 for code generation.

- Evaluation Settings
  - Temperature set to 0.6
  - top-p set to 0.95
  - Maximum generation length is 32,768 for math benchmarks and 40,960 for coding benchmarks.
  - They sample 32 solutions per problem for math benchmarks, and 4 solutions per problem for code, reporting the average pass@1.

- Benefits
  - No live teacher required
  - Lightning OPD achieves a 3.6x speedup at the 4B scale, reducing total GPU hours from 72 to just 20, and a 4x speedup at the 8B scale, bringing the full pipeline down from 120 to just 30 GPU hours.

[8] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, 2024.

[9] Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. https://thinkingmachines.ai/blog/on-policy-distillation.

[10] Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.

[11] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

[12] Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

[13] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

[14] Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, Renjie Pi, Grace Lam, Nayeon Lee, Alexander Bukharin, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation. 2026.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[16] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[17] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[18] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[19] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[20] Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. Rl's razor: Why online reinforcement learning forgets less. *arXiv preprint arXiv:2509.04259*, 2025.

[21] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[22] Jian Hu. REINFORCE++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.

[23] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

[24] Ziniu Li, Tian Xu, Yushun Zhang, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. *arXiv preprint arXiv:2310.10505*, 2023.

[25] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[26] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

[27] MiniMax, Aonian Shan, Bangwei Gong, Bo Yang, et al. MiniMax-M1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

[28] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

[29] Yufeng Yuan, Yu Yue, Ruofei Zhu, Tiantian Fan, and Lin Yan. What's behind PPO's collapse in long-CoT? value optimization holds the secret. *arXiv preprint arXiv:2503.01491*, 2025.

[30] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, Tiantian Fan, Zhengyin Du, Xiangpeng Wei, et al. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.

[31] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of LLMs. *arXiv preprint arXiv:2410.01679*, 2024.

[32] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chengqi Zhao, Chenggang Deng, Chengpeng Zhang, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[33] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, et al. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[34] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.

[35] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

[36] Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

[37] Lu Ma, Hao Liang, Meiyi Qiang, Lexiang Tang, Xiaochen Ma, Zhen Hao Wong, Junbo Niu, Chengyu Shen, Runming He, Yanhao Li, Bin Cui, and Wentao Zhang. Learning what reinforcement learning can't: Interleaved online fine-tuning for hardest questions. *arXiv preprint arXiv:2506.07527*, 2026.

[38] Wenhao Zhang, Yuexiang Xie, Yuchang Sun, Yanxi Chen, Guoyin Wang, Yaliang Li, Bolin Ding, and Jingren Zhou. On-policy RL meets off-policy experts: Harmonizing supervised fine-tuning and reinforcement learning via dynamic weighting. *arXiv preprint arXiv:2508.11408*, 2025.

[39] Liang Chen, Xueting Han, Li Shen, Jing Bai, and Kam-Fai Wong. Beyond two-stage training: Cooperative SFT and RL for LLM reasoning. *arXiv preprint arXiv:2509.06948*, 2025.

[40] Mingyang Liu, Gabriele Farina, and Asuman Ozdaglar. UFT: Unifying supervised and reinforcement fine-tuning. *arXiv preprint arXiv:2505.16984*, 2025.

[41] Zeyu Huang, Tianhao Cheng, Zihan Qiu, Zili Wang, Yinghui Xu, Edoardo M. Ponti, and Ivan Titov. Blending supervised and reinforcement fine-tuning with prefix sampling. *arXiv preprint arXiv:2507.01679*, 2025.

[42] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *NIPS Deep Learning Workshop*, 2015.

[43] Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016.

[44] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.

[45] Tianzhu Ye, Li Dong, Zewen Chi, Xun Wu, Shaohan Huang, and Furu Wei. Black-box on-policy distillation of large language models. *arXiv preprint arXiv:2511.10643*, 2025.

[46] Kun Liang, Clive Bai, Xin Xu, Chenming Tang, Sanwoo Lee, Weijie Liu, Saiyong Yang, and Yunfang Wu. Orbit: On-policy exploration-exploitation for controllable multi-budget reasoning. *arXiv preprint arXiv:2601.08310*, 2026.

[47] Emiliano Penaloza, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged information distillation for language models. *arXiv preprint arXiv:2602.04942*, 2026.

[48] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

[49] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.

[50] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[51] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.

[52] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, 2019.

[53] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[54] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[55] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023.

[56] Daniel Ritter, Owen Oertell, Bradley Guo, Jonathan Chang, Kianté Brantley, and Wen Sun. LLMs can learn to reason via off-policy RL. *arXiv preprint arXiv:2602.19362*, 2026.

[57] Yao Lu, Dengdong Fan, Jianzheng Nie, Fan Xu, Jie Chen, Bin Zhou, and Yonghong Tian. PCL-Reasoner-V1.5: Advancing math reasoning with offline reinforcement learning. *arXiv preprint arXiv:2601.14716*, 2026.

[58] Yaoxiang Wang, Haoling Li, Xin Zhang, Jie Wu, Xiao Liu, Wenxiang Hu, Zhongxin Guo, Yangyu Huang, Ying Xin, Yujiu Yang, Jinsong Su, Qi Chen, and Scarlett Li. Encompassing diversity and complexity in code generation. *arXiv preprint arXiv:2501.04694*, 2025.

[59] AI-MO. AIME 2024. https://huggingface.co/datasets/AI-MO/aimo-validation-aime, 2024.

[60] OpenCompass. AIME 2025. https://github.com/open-compass/opencompass, 2025.

[61] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.

[62] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[63] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024.

[64] THUDM. slime: An SGLang-native post-training framework for RL scaling. https://github.com/THUDM/slime, 2025.

[65] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025.

[66] Yang Chen, Zhuolin Yang, Zihan Liu, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nemotron-cascade: Scaling cascaded reinforcement learning for general-purpose reasoning models. *arXiv preprint arXiv:2512.13607*, 2025.

A. Proofs

We provide complete proofs of all theoretical results stated in Section 3.3. We use Assumptions 3.1–3.3 (and Assumption 3.4 for the teacher-mismatch results) as stated in Section 3.3. We begin with a fundamental preliminary result and then prove each theorem in order.

Notation. Let Ω be a finite vocabulary. A prompt $q \in \Omega^*$ is drawn from $p(q)$. A response $x = (a_1, \ldots, a_T)$ is generated as $\pi_\theta(x \mid q) = \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t)$, where $s_t = (q, a_1, \ldots, a_{t-1})$. The sequence-level IS ratio is $w(x;\theta) := \pi_\theta(x \mid q)/\pi_{\text{ref}}(x \mid q)$.

A.1. Preliminary: Importance-Sampling Decomposition

We begin with a fundamental identity relating the online and offline gradients via IS reweighting. This is used in all subsequent proofs.

Proposition A.1 (IS Decomposition)**.** Under Assumption 3.2, the online gradient equals the IS-reweighted offline expectation:

$$\nabla J_{\text{on}}(\theta) = \mathbb{E}_{x\sim\pi_{\text{ref}}}\left[w(x;\theta) \cdot \sum_{t=1}^{T} A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t) \right].$$

The offline gradient $\nabla J_{\text{off}}(\theta) = \mathbb{E}_{x\sim\pi_{\text{ref}}}[\sum_t A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)]$ is the special case $w \equiv 1$.

Proof. Since Ω is a finite vocabulary, all expectations are finite sums. For any function $f(x)$:

$$\mathbb{E}_{x\sim\pi_\theta}[f(x)] = \sum_x \pi_\theta(x)\, f(x) = \sum_x \pi_{\text{ref}}(x) \cdot \frac{\pi_\theta(x)}{\pi_{\text{ref}}(x)} \cdot f(x) = \mathbb{E}_{x\sim\pi_{\text{ref}}}[w(x;\theta)\, f(x)],$$

where the second equality requires $\pi_{\text{ref}}(x) > 0$ wherever $\pi_\theta(x) > 0$, which holds by Assumption 3.2. Applying this with $f(x) = \sum_t A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)$ (stop-gradient applied to $A_t(\theta)$) gives the result. □

A.2. Proof of Theorem 3.5

Proof. Define the per-trajectory gradient $f(x;\theta) := \sum_{t=1}^{T} A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)$. By Proposition A.1:

$$\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta) = \mathbb{E}_{x\sim\pi_{\text{ref}}}[(w(x;\theta) - 1)\, f(x;\theta)]\,.$$

Applying the Cauchy–Schwarz inequality to the expectation:

$$\|\mathbb{E}_{\pi_{\text{ref}}}[(w-1)\, f]\|_2 \le \sqrt{\mathbb{E}_{\pi_{\text{ref}}}[(w-1)^2]} \cdot \sqrt{\mathbb{E}_{\pi_{\text{ref}}}[\|f\|_2^2]}.$$

First factor. Since $\mathbb{E}_{\pi_{\text{ref}}}[w(x;\theta)] = \sum_x \pi_\theta(x) = 1$:

$$\mathbb{E}_{\pi_{\text{ref}}}\left[(w-1)^2\right] = \mathbb{E}_{\pi_{\text{ref}}}\left[w^2\right] - 2\,\mathbb{E}_{\pi_{\text{ref}}}[w] + 1 = \mathbb{E}_{\pi_{\text{ref}}}\left[w^2\right] - 1 = \chi^2(\pi_\theta \,\|\, \pi_{\text{ref}}).$$

Second factor. By Assumption 3.3, $\|\nabla \log \pi_\theta(a_t \mid s_t)\|_2 \le G$ for each t. By the Cauchy–Schwarz inequality over tokens:

$$\|f(x;\theta)\|_2 \le \left| \sum_{t=1}^{T} A_t(\theta) \right| \cdot G,$$

and therefore:

$$\mathbb{E}_{\pi_{\text{ref}}}\left[\|f(x;\theta)\|_2^2\right] \le G^2 \cdot \mathbb{E}_{\pi_{\text{ref}}}\left[\left(\sum_{t=1}^{T} A_t(\theta) \right)^2 \right] \le G^2\, \sigma_A^2,$$

where the last inequality uses Assumption 3.1.

Combining.

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 \le \sqrt{\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}})} \cdot G\,\sigma_A. \qquad \square$$

□

Corollary A.2 (Zero Gap at Initialization)**.** When $\pi_\theta = \pi_{\text{ref}}$, $\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}}) = 0$, so $\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 = 0$. Under standard smoothness conditions, the gap grows at rate $O(\eta\, k)$ after k gradient steps with step size η.

Proof. At $\pi_\theta = \pi_{\text{ref}}$, $w(x;\theta) = 1$ for all x, so $\mathbb{E}_{\pi_{\text{ref}}}[w^2] = 1$ and $\chi^2 = 0$. The growth rate follows from a first-order Taylor expansion of $\chi^2(\pi_{\theta-\eta g}\|\pi_{\text{ref}})$ around θ. □

Remark on χ^2 versus KL. A bound via Pinsker's inequality yields $\|\nabla J_{\text{on}} - \nabla J_{\text{off}}\|_2 \le MTG\sqrt{2\,\text{KL}(\pi_\theta\|\pi_{\text{ref}})}$, where $M = \sup_t |A_t(\theta)|$. This is vacuous in practice because M diverges whenever $\pi_\theta(a_t|s_t) \to 0$. Theorem 3.5 avoids this by using the L^2 constant $\sigma_A \ll MT$ and replacing KL with χ^2. Although $\chi^2 \ge \text{KL}$, advantage clipping and KL regularization keep χ^2 small throughout training.

A.3. Proof of Theorem 3.7

Proof. We show that the first-order stationarity conditions of J_{on} and J_{off} over Θ are equivalent.

Define the weighted inner product on token-sequence functions:

$$\langle g,\, h\rangle_\mu := \mathbb{E}_{x\sim\mu}\left[\sum_{t=1}^{T} g(a_t, s_t)\, h(a_t, s_t)\right].$$

Let $\mathcal{S}_{\theta^*} = \text{span}\big\{\partial_v \log \pi_{\theta^*}(a_t \mid s_t) : v \in \mathbb{R}^{|\Theta|}\big\}$ be the tangent space of Π_Θ at θ^*.

The stationarity condition $\nabla J_{\text{off}}(\theta^*) = 0$ requires, for every direction $v \in \mathbb{R}^{|\Theta|}$:

$$\langle A(\theta^*),\, \partial_v \log \pi_{\theta^*}\rangle_{\pi_{\text{ref}}} = 0,$$

where we write $A(\theta) = \sum_t A_t(\theta)$ for shorthand. This is precisely the statement that $A(\theta^*)$ is *orthogonal to* $\mathcal{S}_{\theta^*}$ in $L^2(\pi_{\text{ref}})$:

$$\Pi_{\mathcal{S}_{\theta^*}}[A(\theta^*)] = 0,$$

where $\Pi_{\mathcal{S}_{\theta^*}}$ denotes the $L^2(\pi_{\text{ref}})$-orthogonal projection onto $\mathcal{S}_{\theta^*}$.

For standard OPD, the stationarity condition $\nabla J_{\text{on}}(\theta^*) = 0$ requires:

$$\langle A(\theta^*),\, \partial_v \log \pi_{\theta^*}\rangle_{\pi_{\theta^*}} = 0.$$

By Assumption 3.2, π_{ref} and π_{θ^*} are mutually absolutely continuous (the IS weight $w = \pi_{\theta^*}/\pi_{\text{ref}}$ is strictly positive almost everywhere). Therefore the two inner products $\langle\cdot,\cdot\rangle_{\pi_{\text{ref}}}$ and $\langle\cdot,\cdot\rangle_{\pi_{\theta^*}}$ define equivalent topologies on function space: a function orthogonal to $\mathcal{S}_{\theta^*}$ under one measure is orthogonal under the other. Hence the two stationarity conditions are equivalent and J_{on}, J_{off} share the same set of stationary points $\Theta^* \subseteq \Theta$.

To characterise Θ^*: at any stationary point θ^*, the projection condition $\Pi_{\mathcal{S}_{\theta^*}}[A(\theta^*)] = 0$ means no first-order update in Θ can reduce $\|A(\theta^*)\|_{L^2}^2$. Expanding $A_t = \log \pi_T - \log \pi_\theta$, we have $\|A(\theta^*)\|_{L^2}^2 = \text{KL}(\pi_{\theta^*}\|\pi_T) + \text{const}$, so every stationary point is a local minimiser of $\text{KL}(\pi_\theta\|\pi_T)$ over Π_Θ. □

Total error decomposition. Combining Theorems 3.5 and 3.7, the final student satisfies:

$$\text{KL}(\pi_{\theta_{\text{final}}} \,\|\, \pi_T) \le \underbrace{\varepsilon_{\text{approx}}}_{\substack{\text{model capacity}\\ \text{(irreducible)}}} + \underbrace{\varepsilon_{\text{opt}}}_{\substack{\text{optimisation error}\\ (\to 0 \text{ with training})}} + \underbrace{O\Big(G\,\sigma_A\sqrt{\chi^2(\pi_\theta\|\pi_{\text{ref}})}\Big)}_{\text{offline-online distribution gap}},$$

where $\varepsilon_{\text{approx}} = \min_{\theta\in\Theta} \text{KL}(\pi_\theta\|\pi_T)$. Switching from offline to online rollouts reduces only the third term, not $\varepsilon_{\text{approx}}$, which is the dominant term determined by model capacity.

A.4. Proof of Theorem 3.9

Proof. Recall $f(x;\theta) = \sum_t A_t(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)$. By Proposition A.1:

$$\nabla J_{\text{on}}(\theta) = \mathbb{E}_{\pi_{\text{ref}}}[w(x;\theta) \cdot f(x;\theta)].$$

Using the covariance decomposition $\mathbb{E}[wf] = \mathbb{E}[w]\,\mathbb{E}[f] + \text{Cov}[w, f]$ and $\mathbb{E}_{\pi_{\text{ref}}}[w] = 1$:

$$\nabla J_{\text{on}}(\theta) = \mathbb{E}_{\pi_{\text{ref}}}[f] + \text{Cov}_{\pi_{\text{ref}}}[w,\ f] = \nabla J_{\text{off}}(\theta) + \text{Cov}_{\pi_{\text{ref}}}[w(x;\theta),\ f(x;\theta)]\,.$$

Rearranging gives $\nabla J_{\text{off}}(\theta) = \nabla J_{\text{on}}(\theta) - \text{Cov}_{\pi_{\text{ref}}}[w, f]$.

Sign of the covariance term. At $\pi_\theta = \pi_{\text{ref}}$, $w \equiv 1$ is constant so $\text{Cov}[w, f] = 0$ and $\nabla J_{\text{off}} = \nabla J_{\text{on}}$ exactly. As π_θ drifts, w becomes large on trajectories π_θ over-weights relative to π_{ref}; on those same trajectories f also tends to be large, making the covariance positive. The subtracted term therefore grows with drift, producing the restoring-force effect described in Remark 3.9. □

A.5. Proof of Theorem 3.11

We now consider the case where the SFT-stage teacher π_T^{SFT} and the OPD-stage reference teacher π_T^{OPD} differ. The SFT model π_{ref} was trained on data generated by π_T^{SFT}, so its distribution concentrates on trajectories preferred by π_T^{SFT}. However, the advantage function $A_t(\theta) = \log \pi_T^{\text{OPD}}(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t)$ reflects π_T^{OPD}. This mismatch introduces an irreducible additive bias.

Proof. Define $\delta := \chi^2(\pi_T^{\text{SFT}} \| \pi_T^{\text{OPD}})$ and $\Delta_t := \log \pi_T^{\text{SFT}}(a_t \mid s_t) - \log \pi_T^{\text{OPD}}(a_t \mid s_t)$. The advantage under teacher inconsistency decomposes as:

$$A_t(\theta) = \underbrace{\left(\log \pi_T^{\text{SFT}}(a_t \mid s_t) - \log \pi_\theta(a_t \mid s_t)\right)}_{\text{consistent advantage } A_t^{\text{cons}}(\theta)} - \Delta_t.$$

Correspondingly, define $f_{\text{cons}}(x;\theta) = \sum_t A_t^{\text{cons}}(\theta) \cdot \nabla \log \pi_\theta(a_t \mid s_t)$ and $f_\Delta(x;\theta) = -\sum_t \Delta_t \cdot \nabla \log \pi_\theta(a_t \mid s_t)$, so $f = f_{\text{cons}} + f_\Delta$.

By Proposition A.1 and the triangle inequality:

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 = \|\mathbb{E}_{\pi_{\text{ref}}}[(w-1)f]\|_2 \le \underbrace{\|\mathbb{E}_{\pi_{\text{ref}}}[(w-1)f_{\text{cons}}]\|_2}_{\text{(I)}} + \underbrace{\|\mathbb{E}_{\pi_{\text{ref}}}[(w-1)f_\Delta]\|_2}_{\text{(II)}}.$$

Term (I): Consistent part. This is identical to the proof of Theorem 3.5 with the consistent advantage, giving:

$$\text{(I)} \le G\,\sigma_A \cdot \sqrt{\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}})}.$$

Term (II): Mismatch part. Applying Cauchy–Schwarz:

$$\text{(II)} \le \sqrt{\chi^2(\pi_\theta \| \pi_{\text{ref}})} \cdot G \cdot \sqrt{\mathbb{E}_{\pi_{\text{ref}}}\left[\left(\sum_t \Delta_t\right)^2\right]}.$$

Beyond this, even at $\pi_\theta = \pi_{\text{ref}}$ (where $w \equiv 1$ and Term (II) $= 0$), there is a residual bias in the offline gradient itself:

$$\mathbb{E}_{\pi_{\text{ref}}}[f_\Delta] = -\sum_t \mathbb{E}_{\pi_{\text{ref}}}[\Delta_t \cdot \nabla \log \pi_\theta(a_t \mid s_t)] \ne 0 \quad \text{when } \delta > 0.$$

By Assumption 3.3 and Cauchy–Schwarz:

$$\|\mathbb{E}_{\pi_{\text{ref}}}[f_\Delta]\|_2 \le G \cdot \sqrt{\mathbb{E}_{\pi_{\text{ref}}}\left[\left(\sum_t \Delta_t\right)^2\right]} \le G\,\sigma_\Delta,$$

where the last inequality applies Assumption 3.4 directly. Adding this residual to Term (I):

$$\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{off}}(\theta)\|_2 \leq G \cdot \left(\sigma_A \sqrt{\chi^2(\pi_\theta \,\|\, \pi_{\text{ref}})} + \sigma_\Delta\right).$$

When $\sigma_\Delta > 0$, the term $G\,\sigma_\Delta$ is irreducible: it persists even at $\pi_\theta = \pi_{\text{ref}}$ (where $\chi^2(\pi_\theta \| \pi_{\text{ref}}) = 0$) and cannot be reduced by any amount of additional training or data. □

A.6. Proof of Theorem 3.13

Proof. Let $\nabla J_{\text{on}}^{\delta=0}(\theta)$ denote the standard OPD gradient under a consistent teacher ($\pi_T^{\text{SFT}} = \pi_T^{\text{OPD}}$), and let $\nabla J_{\text{on}}(\theta)$ denote the gradient under the mismatched teacher $\pi_T^{\text{OPD}} \neq \pi_T^{\text{SFT}}$. For any iterate θ reachable from π_{ref}, their difference is:

$$\begin{aligned}\nabla J_{\text{on}}(\theta) - \nabla J_{\text{on}}^{\delta=0}(\theta) &= \mathbb{E}_{x\sim\pi_\theta}\left[\sum_t \left(A_t^{\text{OPD}}(\theta) - A_t^{\text{SFT}}(\theta)\right) \cdot \nabla \log \pi_\theta(a_t \mid s_t)\right] \\ &= -\mathbb{E}_{x\sim\pi_\theta}\left[\sum_t \Delta_t \cdot \nabla \log \pi_\theta(a_t \mid s_t)\right],\end{aligned}$$

where $\Delta_t = \log \pi_T^{\text{SFT}}(a_t \mid s_t) - \log \pi_T^{\text{OPD}}(a_t \mid s_t)$.

By Assumption 3.3 and the Cauchy–Schwarz inequality:

$$\left\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{on}}^{\delta=0}(\theta)\right\|_2 \leq G \cdot \sqrt{\mathbb{E}_{x\sim\pi_\theta}\left[\left(\sum_t \Delta_t\right)^2\right]}.$$

Since θ is reachable from π_{ref}, by Assumption 3.2 $\pi_\theta \ll \pi_{\text{ref}}$. Applying the IS identity (Proposition A.1):

$$\mathbb{E}_{x\sim\pi_\theta}\left[\left(\sum_t \Delta_t\right)^2\right] = \mathbb{E}_{x\sim\pi_{\text{ref}}}\left[w(x;\theta)\left(\sum_t \Delta_t\right)^2\right].$$

At or near initialisation ($\pi_\theta \approx \pi_{\text{ref}}$, $w \approx 1$), this reduces to $\mathbb{E}_{\pi_{\text{ref}}}[(\sum_t \Delta_t)^2] \leq \sigma_\Delta^2$ by Assumption 3.4. Substituting:

$$\left\|\nabla J_{\text{on}}(\theta) - \nabla J_{\text{on}}^{\delta=0}(\theta)\right\|_2 \leq G\,\sigma_\Delta.$$

This bound is independent of the rollout distribution: the same irreducible term $G\,\sigma_\Delta$ corrupts the gradient of standard OPD whenever $\sigma_\Delta > 0$, causing it to converge to a fixed point shifted away from the capacity-limited optimum identified in Theorem 3.7. □

B. Implementation Details

Tables 5 and 6 list the full hyperparameter configurations for the SFT and OPD stages respectively. For the OPD stage, the same hyperparameters are used for both the math and code domains. For code generation, however, the OPD stage is initialized from the math-trained OPD checkpoint rather than the SFT checkpoint directly. We find this consistently outperforms initializing code OPD from the SFT model, consistent with prior findings that math reasoning training provides a stronger initialization for code training [66].

Evaluation Templates. We use the following prompt templates for evaluation. For mathematical reasoning benchmarks, the prompt follows the Qwen3 chat format:

```
<|im_start|>user
Question: {problem}
Please reason step by step, and put your final answer within \boxed{}.
<|im_end|>
<|im_start|>assistant
```

Table 5 | SFT stage hyperparameters.

Hyperparameter	**4B Scale**	**8B Scale**
Training steps	3000	3000
Global batch size	256	128
Max sequence length	16384	16384
Learning rate	8×10^{-5}	8×10^{-5}
LR schedule	cosine	cosine
Warmup ratio	0.1	0.1
Packing	✓	✓
DeepSpeed stage	ZeRO-0	ZeRO-1

Table 6 | OPD stage hyperparameters for standard OPD and Lightning OPD. The same configuration is used for both the math and code domains.

Hyperparameter	**4B Scale**	**8B Scale**
Training steps	150	150
Global batch size	256	256
Max response length	4096	4096
Learning rate	2×10^{-6}	2×10^{-6}
LR schedule	constant	constant
Weight decay	0.1	0.1
Adam β_1	0.9	0.9
Adam β_2	0.98	0.98
Rollout temperature	0.8	0.8
Rollout top-p	1.0	1.0
Tensor parallel size	2	4

For code generation benchmarks, the prompt includes a system message and a task description following LiveCodeBench conventions:

```
<|im_start|>system
You are a helpful and harmless assistant. You are Qwen developed by Alibaba.
You should think step-by-step.
<|im_end|>
<|im_start|>user
You will be given a question (problem specification) and will generate a correct
Python program that matches the specification and passes all tests.

Question: {question}

Read the inputs from stdin solve the problem and write the answer to stdout
(do not directly test on the sample inputs). Enclose your code within delimiters
as follows. Ensure that when the python program runs, it reads the inputs, runs
the algorithm and writes output to STDOUT.
# YOUR CODE HERE

<|im_end|>
<|im_start|>assistant
```